Argo CD, Keycloak and Active Directory SSO

Complete conversation export · 14–15 October 2026 · Asia/Kolkata

> **You** · Wed 14 Oct 2026, 07:21
> argocd installation using the helm

CLAUDE

Here's the standard way to install Argo CD with Helm.

1. Add the repo

```
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update
```

2. Install into its own namespace

```
helm install argocd argo/argo-cd \
  --namespace argocd \
  --create-namespace
```

To pin a chart version (recommended for anything beyond a lab), check available versions and add `--version`:

```
helm search repo argo/argo-cd --versions | head
helm install argocd argo/argo-cd -n argocd --create-namespace --version <chart-version>
```

3. Verify

```
kubectl get pods -n argocd
kubectl get svc -n argocd
```

4. Get the initial admin password

```
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d; echo
```

The username is `admin`.

5. Access the UI

Quick test via port-forward:

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Then open https://localhost:8080.

Exposing it properly (values file)

For a real cluster, use a `values.yaml` instead of CLI flags. A typical starting point with ingress and TLS terminated at the ingress:

```yaml
global:
  domain: argocd.example.com

configs:
  params:
    server.insecure: true   # TLS terminated at ingress

server:
  ingress:
    enabled: true
    ingressClassName: nginx
    tls: true
    annotations:
      cert-manager.io/cluster-issuer: letsencrypt-prod

controller:
  replicas: 1

redis-ha:
  enabled: false   # set true with replicas >= 3 for HA
```

Install or upgrade with it:

```
helm upgrade --install argocd argo/argo-cd \
  -n argocd --create-namespace \
  -f values.yaml
```

If your cluster has no ingress controller (common on a fresh VKS cluster), use a LoadBalancer instead:

```yaml
server:
  service:
    type: LoadBalancer
```

Other useful notes

- **HA setup:** set `redis-ha.enabled: true`, `controller.replicas: 1` (the controller shards via `ARGOCD_CONTROLLER_REPLICAS`), and `server.replicas: 2`, `repoServer.replicas: 2`, `applicationSet.replicas: 2`.
- **Air-gapped or private registry:** override `global.image.repository` and `global.imagePullSecrets`, and mirror the Redis and Dex images too.
- **Uninstall:** `helm uninstall argocd -n argocd`. CRDs are kept by default; remove them manually with `kubectl delete crd applications.argoproj.io applicationsets.argoproj.io appprojects.argoproj.io` if you want a clean slate.
- **Change the admin password:** after login, run `argocd account update-password`, then delete `argocd-initial-admin-secret`.

If you tell me your environment (ingress controller, HA needs, private registry, SSO), I can tailor the values file.

> **You** · Wed 14 Oct 2026, 07:21
> share the most advance installation

CLAUDE

I'll put together a production-grade values file (HA, SSO, autoscaling, metrics, notifications) and send it over.

📎 **argocd-values-advanced.yaml** attached (full contents in Appendix A).

The values file `argocd-values-advanced.yaml` is attached. It covers:

- **HA:** 2 replicas for the server, repo-server, ApplicationSet and controller, with HPAs, PDBs and hard pod anti-affinity. Redis runs as Redis-HA with HAProxy.
- **Sharded controller:** `controller.replicas: 2` makes the chart set `ARGOCD_CONTROLLER_REPLICAS` automatically, so clusters are split across the controller pods.
- **SSO and RBAC:** direct OIDC with Dex disabled, group-based RBAC, a readonly default role and the web terminal off. Switch `admin.enabled` to false once SSO works.
- **Observability:** ServiceMonitors on every component, plus PrometheusRule alerts for out-of-sync and unhealthy apps.
- **Notifications:** Slack triggers for sync failure and degraded health, with the token coming from an externally managed secret.
- **Other:** NetworkPolicies, CRDs kept on uninstall, annotation-based resource tracking, apps-in-any-namespace and progressive syncs.

Deploy it safely

```
        helm repo add argo https://argoproj.github.io/argo-helm && helm repo update
        helm search repo argo/argo-cd --versions | head          # pick and pin a version

        helm show values argo/argo-cd --version <ver> > defaults.yaml   # diff key names against my file
        helm template argocd argo/argo-cd -n argocd --version <ver> -f argocd-values-advanced.yaml | kubectl apply --
dry-run=server -f -

        helm upgrade --install argocd argo/argo-cd \
          -n argocd --create-namespace \
          --version <ver> \
          -f argocd-values-advanced.yaml \
          --atomic --wait --timeout 10m
```

- **Pin and diff:** I wrote the keys from the current chart structure, but they shift between major chart versions, so diff against `defaults.yaml` for your pinned version.
- **Secrets:** the OIDC client secret, Slack token and repo credentials should come from External Secrets or Sealed Secrets, not from the values file.
- **Self-management:** once it's running, bootstrap a root Application (app-of-apps or ApplicationSet) that points at this chart and values in Git, so Argo CD manages its own upgrades.

I also need to correct my earlier note on HA. The controller doesn't have to stay at one replica. It scales by sharding, and the chart wires that up from `controller.replicas`.

I can add an app-of-apps bootstrap, an AppProject and RBAC layout, or multi-cluster registration if you want those next.

> **You** · Wed 14 Oct 2026, 07:24
> I have an windows active directory in my lab setup and i want to use the OIDC as keyclock and than i want to integrate OIDC with Active Directory is this possible to perform

CLAUDE

Yes, this is a standard setup. Keycloak acts as the OIDC provider for Argo CD and uses your Active Directory as the user store, so the chain is Argo CD → Keycloak (OIDC) → AD (LDAP).

Option 1: LDAP user federation (recommended for a lab)

Keycloak reads users and groups directly from AD, and users log in with their AD username and password.

1. **Create the federation:** in your realm, go to *User federation* → *Add LDAP provider* and set:

- Vendor: Active Directory
- Connection URL: `ldaps://dc01.lab.local:636`. Use LDAPS rather than plain 389, and import your AD CA certificate into Keycloak's truststore.
- Users DN: e.g. `OU=Users,DC=lab,DC=local`
- Username LDAP attribute: `sAMAccountName`
- RDN LDAP attribute: `cn`
- UUID LDAP attribute: `objectGUID`
- User object classes: `person, organizationalPerson, user`
- Bind DN and credential: a read-only service account
- Edit mode: `READ_ONLY`

2. **Test and sync:** use *Test connection*, *Test authentication*, then *Synchronize all users*.
3. **Add a group mapper:** under the provider's *Mappers*, add a `group-ldap-mapper`:
   - LDAP Groups DN: e.g. `OU=Groups,DC=lab,DC=local`
   - Group object classes: `group`
   - Membership attribute: `member`
   - Mode: `READ_ONLY`
   - User groups retrieve strategy: `LOAD_GROUPS_BY_MEMBER_ATTRIBUTE_RECURSIVELY`, if you use nested AD groups
4. **Create the Argo CD client:** in the same realm, create an OIDC client named `argocd`:
   - Client authentication: on
   - Valid redirect URI: `https://argocd.example.com/auth/callback`
   - Post-logout redirect URI: `https://argocd.example.com/applications`
5. **Send groups in the token:** create a client scope called `groups` with a *Group Membership* mapper, set the claim name to `groups`, turn *Full group path* off, and add it to ID token and userinfo. Attach the scope to the `argocd` client.
6. **Wire up Argo CD:** the `oidc.config` in my values file already points to Keycloak. Set the issuer to `https://<keycloak-host>/realms/<realm>`, set `clientID: argocd`, and put the client secret in the externally managed secret.
7. **Map AD groups to roles:** the group names from AD become the subjects in Argo CD RBAC, for example:

```
      configs:
        rbac:
          scopes: "[groups]"
          policy.csv: |
            g, ArgoCD-Admins, role:admin
            g, ArgoCD-Deployers, role:deployer
```

Option 2: Kerberos/SPNEGO (optional add-on)

If your users sit on domain-joined Windows machines, Keycloak can also do silent SSO with Kerberos. This needs a service principal and keytab for Keycloak plus browser configuration, so I'd get Option 1 working first.

Option 3: Brokering through AD FS (only if you already run it)

Keycloak can add AD FS as an external identity provider over OIDC or SAML. It's more moving parts than LDAP federation, so I wouldn't set it up just for this.

Things that commonly break

- **LDAPS trust:** Keycloak must trust the DC's certificate chain. A missing CA import is the most common failure.
- **Keycloak certificate trust from Argo CD:** if Keycloak uses a lab CA or self-signed certificate, Argo CD can't verify it and login fails. Add the CA with `rootCA` in `oidc.config`, or mount it into the Argo CD server.
- **Reachability:** the Argo CD pods must be able to reach the Keycloak issuer URL, not just your browser. Use a hostname that resolves inside the cluster.
- **Missing groups:** if the `groups` claim is empty, check that the client scope is attached, the Group Membership mapper is enabled for the ID token, and the user's groups appear under *Users* → *Groups* in Keycloak.
- **CLI login:** use `argocd login argocd.example.com --sso --grpc-web` because TLS terminates at the ingress.

I can also write the Keycloak side for you, as Helm values or a realm import JSON with the LDAP provider, mappers and the `argocd` client pre-configured. Let me know which you prefer.

> **You** · Wed 14 Oct 2026, 07:30
> share the steps to install keyclock on a docker

CLAUDE

I'll write a Docker Compose file for Keycloak with Postgres and TLS, set up so it works with the AD and Argo CD integration we just discussed.

📎 **docker-compose.yml** attached (the later, persistent version is in Appendix B).

The `docker-compose.yml` is attached. It runs Keycloak in production mode with Postgres and HTTPS. Here is how to bring it up.

1. Prerequisites

- Docker Engine with the Compose plugin (`docker compose version`).
- Port 443 free on the host. If it isn't, change the mapping to `"8443:8443"` and use `https://keycloak.lab.local:8443` as `KC_HOSTNAME`.
- A DNS record or hosts entry for `keycloak.lab.local` pointing to the Docker host. Your AD DNS is the best place for it, so that the Argo CD pods and your browser resolve the same name.

2. Create the folder and certificate

```
mkdir -p keycloak/certs keycloak/truststore && cd keycloak
# copy docker-compose.yml here

openssl req -x509 -newkey rsa:4096 -sha256 -days 825 -nodes \
  -keyout certs/tls.key -out certs/tls.crt \
  -subj "/CN=keycloak.lab.local" \
  -addext "subjectAltName=DNS:keycloak.lab.local"

chmod 644 certs/tls.key   # lab only: the container runs as uid 1000 and must read it
```

If you have an internal CA (for example AD Certificate Services), issue the certificate from it instead. Clients then only need to trust one CA.

3. Add the AD CA to the truststore

Export your AD CA certificate as PEM and drop it in, so LDAPS works later:

```
cp /path/to/ad-root-ca.crt truststore/
```

4. Create the `.env` file

```
cat > .env <<EOF
POSTGRES_PASSWORD=$(openssl rand -base64 24 | tr -d '/+=')
KEYCLOAK_ADMIN_PASSWORD=ChangeMe-Strong-Password-123
EOF
chmod 600 .env
```

5. Start it

```
docker compose up -d
docker compose logs -f keycloak
```

Wait for the log line saying Keycloak started and is listening on 8443. The first start takes about a minute because it builds the DB schema.

6. Verify

```
curl -k https://keycloak.lab.local/realms/master/.well-known/openid-configuration
```

You should get JSON. Then open `https://keycloak.lab.local/admin` and log in as `admin` with the password from `.env`.

7. After first login

- Don't configure the `master` realm for apps. Create a new realm (for example `lab`), and the issuer becomes `https://keycloak.lab.local/realms/lab`.
- Create a permanent admin user in `master`, then delete the bootstrap `admin` user.
- Continue with the LDAP federation, `groups` scope and `argocd` client from my previous message.

Useful notes

- **Self-signed certificate:** Argo CD must trust it, so put the contents of `tls.crt` under `rootCA:` in `oidc.config`, and make sure the pods resolve `keycloak.lab.local`.
- **Reverse proxy:** if you later put Keycloak behind nginx or HAProxy, switch to `KC_PROXY_HEADERS=xforwarded`, `KC_HTTP_ENABLED=true`, and terminate TLS at the proxy.
- **Backups:** the data lives in the `pgdata` volume. Back it up with `docker compose exec postgres pg_dump -U keycloak keycloak > keycloak.sql`.
- **Upgrades:** change the image tag, then run `docker compose pull && docker compose up -d`. Back up the database first, since Keycloak migrates the schema on startup.
- **Quick throwaway test:** `docker run -p 8080:8080 -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e KC_BOOTSTRAP_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak:26.4 start-dev` works for a 5-minute look, but don't use it for the Argo CD integration.

Once it's up, I can write a realm import JSON for you with the LDAP provider, group mapper and `argocd` client pre-configured.

> **You** · Wed 14 Oct 2026, 14:27
> how can i export ldap certificate from the windows active directory

CLAUDE

You need the **CA certificate that issued the domain controller's LDAPS certificate**, not the DC's own certificate. Keycloak trusts the CA, and that covers every DC the CA has issued certificates to. Here are the ways to get it, best first.

Method 1: Export the CA certificate from AD Certificate Services (recommended)

On the CA server or any domain-joined machine, in PowerShell or cmd:

```
certutil -ca.cert C:\temp\ad-root-ca.cer
```

If that can't find the CA, list the CAs and target one explicitly:

```
certutil -config - -ping
certutil -config "CA-HOST\CA-Name" -ca.cert C:\temp\ad-root-ca.cer
```

Alternatively, open `http://<ca-host>/certsrv`, choose **Download a CA certificate, certificate chain, or CRL**, select **Base 64**, and click **Download CA certificate**.

Method 2: Export from the Windows certificate store

1. Run `certlm.msc` (or `certmgr.msc` on a domain-joined machine).
2. Go to **Trusted Root Certification Authorities → Certificates**, or **Intermediate Certification Authorities** if you have a two-tier PKI.
3. Right-click your CA certificate, choose **All Tasks → Export**, and pick **Base-64 encoded X.509 (.CER)**.
4. Export every certificate in the chain: the root, plus any intermediates.

Method 3: Pull what the DC actually presents (from Linux/Docker host)

```
openssl s_client -connect dc01.lab.local:636 -showcerts </dev/null
```

Copy each `-----BEGIN CERTIFICATE-----` to `-----END CERTIFICATE-----` block into its own `.crt` file. The first block is the DC's certificate. The later ones are the issuing CA and the root. Keep the CA ones. The DC's own certificate also works if you want to trust that single server only, but it breaks when the certificate renews.

Convert to PEM if needed

Keycloak wants PEM (text starting with `-----BEGIN CERTIFICATE-----`). If your export is DER (binary), convert it:

```
openssl x509 -inform der -in ad-root-ca.cer -out ad-root-ca.crt
```

or on Windows:

```
certutil -encode C:\temp\ad-root-ca.cer C:\temp\ad-root-ca.crt
```

Install it in Keycloak

```
cp ad-root-ca.crt keycloak/truststore/
cd keycloak && docker compose restart keycloak
```

Then in Keycloak, use `ldaps://dc01.lab.local:636` in the LDAP provider and run **Test connection**.

Check that LDAPS works at all

- **DC certificate:** on the DC, run `certlm.msc` and check **Personal → Certificates** for a certificate issued from the *Domain Controller*, *Domain Controller Authentication* or *Kerberos Authentication* template. If none exists, LDAPS isn't enabled yet. Enroll one or enable auto-enrollment, and the DC picks it up without a reboot.
- **Port and certificate:** on Windows, `ldp.exe` → Connection → Connect to `dc01.lab.local`, port 636, SSL ticked. It should connect and show the certificate.
- **Hostname match:** the name in Keycloak's Connection URL must match a SAN on the DC's certificate. Use the FQDN, not an IP address, or you get hostname verification errors.
- **Chain:** if the CA is two-tier, put both the root and the intermediate into `truststore/`.
- **Reachability:** the Docker host must be able to reach the DC on TCP 636.

If **Test connection** still fails after this, send me the Keycloak log line (`docker compose logs keycloak | tail -50`) and I'll help you work out which of these it is.

> **You** · Wed 14 Oct 2026, 14:37
> go-yaml load error in parser (while parsing a block mapping) at L25.C5-L43.C7: did not find expected key

CLAUDE

Running the YAML check on both files now.

Both files I sent parse cleanly: `docker-compose.yml` and `argocd-values-advanced.yaml` both returned OK here, so the copy on your machine differs from what I delivered.

Your error range (L25.C5 to L43.C7) matches the `keycloak:` service in `docker-compose.yml`. Line 25 is `image:` at 4 spaces of indent, and the failure is at line 43, `KC_HTTPS_CERTIFICATE_FILE`, at 6 spaces. "Did not find expected key" at that point usually means the file was altered when copied from chat. A line got dedented, a tab replaced spaces, or a long line wrapped and broke the mapping.

To fix it

1. Use the attached `docker-compose.yml` file as delivered instead of copy-pasting from the chat.
2. If you already pasted it, check the whitespace on the failing lines:

```
sed -n '40,45p' docker-compose.yml | cat -A
```

Each line under `environment:` should start with exactly 6 spaces, with no `^I` (tab) characters and no line ending in `\`.

3. Validate before starting:

```
docker compose config
```

If it still fails, paste the output of the `cat -A` command above and I'll point to the exact line.

> **You** · Wed 14 Oct 2026, 14:39
> go-yaml load error in parser (while parsing a block mapping) at L25.C5-L43.C7: did not find expected key

CLAUDE

Same error, so the copy on your side still differs from mine. I'll send a stripped-down version with no comments or blank lines inside the `environment` block, since pasting often breaks those.

📎 **docker-compose-clean.yml** attached.

I've sent `docker-compose-clean.yml`, which is the same Keycloak and Postgres setup with every comment and blank line removed. It parses cleanly here and contains no tabs.

Download it and save it as `docker-compose.yml` instead of copy-pasting. If you do paste, use a plain-text editor, not a rich-text box, because those can change the indentation.

Then check it before starting:

```
docker compose config
```

If you still get `L25.C5-L43.C7`, the error is coming from your tool or editor rather than the file. Tell me where you see it (Portainer, Dockge, Docker Desktop, or the CLI), and paste the output of:

```
sed -n '24,44p' docker-compose.yml | cat -A
```

That shows the exact whitespace on the lines the error points at.

You · Wed 14 Oct 2026, 18:15

I want to leverage keycloak as an OIDC provider and behind i want to leverage the Active Diretory for the user authentication and group. My keycloak installation is done . share me the step by step instruction on how to do this integration

CLAUDE

Here is the full flow. Hostnames like `dc01.lab.local`, `keycloak.lab.local`, `argocd.example.com`, the realm `lab` and the domain `lab.local` are placeholders, so swap in yours.

The chain: Argo CD → Keycloak (OIDC) → Active Directory (LDAPS). Users log in with their AD credentials, and their AD groups arrive in the token as a `groups` claim.

Part A: Prepare Active Directory

1. **Create a read-only service account** for Keycloak, for example `svc-keycloak`, as a normal domain user with no special rights and "password never expires" set.
2. **Create the groups** Argo CD will use, for example `ArgoCD-Admins` and `ArgoCD-Deployers`, and add your test users to them.
3. **Note the DNs you'll need** by running this on a DC:

```
Get-ADUser svc-keycloak | Select DistinguishedName
Get-ADGroup ArgoCD-Admins | Select DistinguishedName
Get-ADDomain | Select DistinguishedName
```

4. **Confirm LDAPS works** from the Docker host:

```
openssl s_client -connect dc01.lab.local:636 -CAfile truststore/ad-root-ca.crt </dev/null | grep "Verify return code"
```

You want `Verify return code: 0 (ok)`. If you see an error, fix the CA export and truststore first (see my earlier message), then run `docker compose restart keycloak`.

Part B: Create the realm

Log in to `https://keycloak.lab.local/admin`, open the realm dropdown, and choose **Create realm**. Name it `lab`. Don't use `master` for applications.

Part C: Add the AD user federation

Go to **User federation → Add provider → LDAP** and set:

Setting	Value
UI display name	`ad-lab`
Vendor	Active Directory
Connection URL	`ldaps://dc01.lab.local:636`
Enable StartTLS	Off
Use Truststore SPI	Always (or "Only for ldaps")
Bind type	simple
Bind DN	`CN=svc-keycloak,OU=ServiceAccounts,DC=lab,DC=local`
Bind credentials	the service account password
Edit mode	READ_ONLY
Users DN	`OU=Users,DC=lab,DC=local` (or the domain root if users are spread around)
Username LDAP attribute	`sAMAccountName`
RDN LDAP attribute	`cn`
UUID LDAP attribute	`objectGUID`
User object classes	`person, organizationalPerson, user`
Search scope	Subtree
Import users	On
Sync registrations	Off

Optional but useful: set **Custom user LDAP filter** to skip disabled accounts:

```
(&(objectCategory=person)(objectClass=user)(!(userAccountControl:1.2.840.113556.1.4.803:=2)))
```

Then click **Test connection** and **Test authentication**. Both must pass before you save. After saving, use **Action → Sync all users** (and set a periodic sync if you like).

Part D: Map AD groups into Keycloak

1. Open the `ad-lab` provider, go to the **Mappers** tab, and choose **Add mapper → group-ldap-mapper**.
2. Set:
   - Name: `ad-groups`
   - LDAP Groups DN: `OU=Groups,DC=lab,DC=local`
   - Group name LDAP attribute: `cn`
   - Group object classes: `group`
   - Membership LDAP attribute: `member`
   - Membership attribute type: DN
   - Mode: READ_ONLY
   - User groups retrieve strategy: `LOAD_GROUPS_BY_MEMBER_ATTRIBUTE_RECURSIVELY` (so nested AD groups resolve)
   - Drop non-existing groups during sync: On
3. Save, then click **Sync LDAP groups to Keycloak**.
4. Check **Groups** in the left menu to see your AD groups, and **Users → (a test user) → Groups** to confirm membership.

Part E: Put groups into the token

1. Go to **Client scopes → Create client scope**, name it `groups`, type **Default**, protocol OpenID Connect, and save.

2. Open it, go to **Mappers → Configure a new mapper → Group Membership**, and set:

- Name: `groups`
- Token claim name: `groups`
- Full group path: **Off** (so you get `ArgoCD-Admins`, not `/ArgoCD-Admins`)
- Add to ID token: On
- Add to access token: On
- Add to userinfo: On

Part F: Create the Argo CD client

1. Go to **Clients → Create client** and choose OpenID Connect with Client ID `argocd`.
2. On the capability screen, turn **Client authentication** on and keep **Standard flow** checked.
3. Set the URLs:
   - Root URL: `https://argocd.example.com`
   - Valid redirect URIs: `https://argocd.example.com/auth/callback`
   - Valid post logout redirect URIs: `https://argocd.example.com/applications`
   - Web origins: `https://argocd.example.com`
4. Open the client's **Client scopes** tab and make sure `groups` is assigned (add it if not).
5. Open the **Credentials** tab and copy the **Client secret**.

Part G: Verify the token before touching Argo CD

Go to **Clients → argocd → Client scopes → Evaluate**, pick an AD user, and open **Generated ID token**. You should see:

```
"preferred_username": "jdoe",
"email": "jdoe@lab.local",
"groups": ["ArgoCD-Admins"]
```

If `groups` is missing, recheck Parts D and E before continuing.

Part H: Configure Argo CD

Update your values file (if Keycloak uses a lab or self-signed CA, include `rootCA`):

```
configs:
  cm:
    url: https://argocd.example.com
    oidc.config: |
      name: Keycloak
      issuer: https://keycloak.lab.local/realms/lab
      clientID: argocd
      clientSecret: $oidc.keycloak.clientSecret
      requestedScopes: ["openid", "profile", "email", "groups"]
      rootCA: |
        -----BEGIN CERTIFICATE-----
        ...your Keycloak/lab CA certificate...
        -----END CERTIFICATE-----
  rbac:
    scopes: "[groups]"
    policy.default: role:readonly
    policy.csv: |
      g, ArgoCD-Admins, role:admin
      g, ArgoCD-Deployers, role:deployer
```

Provide the secret without committing it to Git:

```
helm upgrade --install argocd argo/argo-cd -n argocd \
  -f argocd-values-advanced.yaml \
  --set-string configs.secret.extra."oidc\.keycloak\.clientSecret"='<client-secret>'
```

Keep `admin.enabled` true until SSO works, then turn it off.

Part I: Test

1. Open `https://argocd.example.com` and click **Log in via Keycloak**.
2. Sign in with an AD account such as `jdoe`. You should land in Argo CD with the permissions of the AD group.
3. CLI: `argocd login argocd.example.com --sso --grpc-web`.

If something fails

Symptom	Likely cause
Test connection fails	LDAPS port blocked, DC certificate has no SAN for the hostname you used, or the CA isn't in `truststore/`
Test authentication fails	Wrong Bind DN or password
Login works but no groups	`groups` scope not attached to the client, or the group mapper hasn't synced
`x509: certificate signed by unknown authority` in Argo CD	Missing `rootCA` in `oidc.config`
Argo CD "failed to query provider"	Pods can't resolve `keycloak.lab.local` (use CoreDNS forwarding or hostAliases)
Invalid redirect URI	Redirect URI in the client doesn't exactly match `https://argocd.example.com/auth/callback`
Logged in but read-only	The group name in `policy.csv` doesn't exactly match the token's group value

Once this works, I can write a Keycloak realm import JSON so the whole setup (LDAP provider, mappers, `groups` scope and `argocd` client) can be recreated with one command.

> **You** · Thu 15 Oct 2026, 03:48
> in the docker compose script modify in such a way that even after the container restart the settings should stay

CLAUDE

Your realm settings (AD federation, mappers, the `argocd` client) are stored in Postgres, so what matters is that the Postgres data survives. I'll pin the volume names so it can't silently change, and add a backup service.

📎 **docker-compose-persistent.yml** attached (full contents in Appendix B).

I've sent `docker-compose-persistent.yml`. The YAML parses cleanly, but I couldn't run `docker compose config` here.

What changed

- **Pinned volume names:** `name: keycloak_pgdata` (database) and `keycloak_data`. Compose normally prefixes volume names with the folder name. If you ever run it from a different folder, you get an empty new database, and that looks exactly like "my settings are gone".
- **`name: keycloak`** at the top fixes the project name for the same reason.
- **Daily backup:** a `db-backup` service writes a `pg_dump` into `./backups` every 24 hours and deletes dumps older than 7 days.
- **Restart policy:** `restart: unless-stopped` was already set, so the containers come back after a host reboot.

Before you switch, keep your current settings

Your existing configuration is in your old Postgres volume, and that volume may not be named `keycloak_pgdata`.

1. Find it:

```
docker volume ls | grep pgdata
```

2. Take a backup:

```
docker compose exec postgres pg_dump -U keycloak -Fc keycloak > keycloak-before-switch.dump
```

3. If the existing volume is, for example, `keycloak_pgdata`, you're set. If it has a different name (say `myfolder_pgdata`), put that exact name in the `pgdata:` block of the new file, so Compose reuses it and nothing is lost.
4. Apply it:

```
docker compose up -d
```

How to keep the settings

- `docker compose down` and `docker compose restart` keep your data.
- `docker compose down -v` deletes the volumes and wipes everything, so avoid `-v`.
- If your earlier run used `start-dev` or had no Postgres volume, the data lived inside the container and a recreate loses it. In that case you'd have to redo the AD setup once.

Restore from a backup if needed

```
docker compose exec -T postgres pg_restore -U keycloak -d keycloak --clean --if-exists < keycloak-before-switch.dump
```

Extra safety: once the AD, group and `argocd` client setup is finished, export the realm (**Realm settings → Action → Partial export**, or `kc.sh export --realm lab`). That JSON can rebuild the whole config on a fresh Keycloak. I can turn it into an import-ready realm file for you if you'd like.

> **You** · Thu 15 Oct 2026, 04:19
> *(attached excerpt: "Part G: Verify the token before touching Argo CD")*
> if i click on evalute i get an unknown error in the browser and when i relogin all the settings are gone

CLAUDE

These are probably one problem. The `Evaluate` error may just be a symptom of Keycloak starting against a fresh, empty database. I can't see your Docker host, so these checks will show which case you're in.

Step 1: Find out whether Keycloak lost its database

```
docker logs keycloak 2>&1 | grep -iE "Initializing master realm|Created temporary admin|development mode"
```

- **"Initializing master realm" appears after your latest restart:** Keycloak started on an empty database, so your settings were not deleted, Keycloak just isn't looking at the data that has them.
- **"development mode" appears:** the container is running `start-dev`, which keeps data inside the container and loses it on recreate. Check `docker inspect keycloak --format '{{.Config.Cmd}}'`, which should show `start`.

Step 2: Check which volume Postgres is using, and what exists

```
docker volume ls
docker inspect keycloak-db --format '{{range .Mounts}}{{.Name}} -> {{.Destination}}{{"\n"}}{{end}}'
```

```
docker exec keycloak-db psql -U keycloak -d keycloak -c "select name, enabled from realm;"
```

The last command should list `master` and `lab`. If only `master` shows, the data isn't in the database being used now.

My most likely guess is that the pinned volume name in `docker-compose-persistent.yml` (`keycloak_pgdata`) differs from the volume your first run created. Compose names that one `<folder>_pgdata`. If so, you now have two volumes, a new empty one in use and your old one untouched.

Step 3: Recover from the old volume

Look for a second volume in `docker volume ls`, such as `something_pgdata`. Confirm it has your data (read-only):

```
docker run --rm -v OLD_VOLUME_NAME:/var/lib/postgresql/data:ro postgres:17 ls /var/lib/postgresql/data
```

Seeing `PG_VERSION` and `base` means it holds a database. Then point the compose file at it:

```
volumes:
  pgdata:
    name: OLD_VOLUME_NAME
```

```
docker compose down        # no -v
docker compose up -d
```

Your `lab` realm, AD federation and `argocd` client come back as they were.

If no old volume exists

If `docker volume ls` shows only the new one and you have no `.dump` file, the earlier data was never stored persistently, probably because it ran under `start-dev` or without a volume. You'll have to redo the AD setup once. With the persistent file it will survive restarts from then on. As soon as it's working, take a realm export (**Realm settings → Action → Partial export**) so it's recoverable in minutes.

Step 4: The "unknown error" on Evaluate

Fix persistence first, then retest, because it may disappear. If it doesn't:

1. Run `docker logs keycloak --tail 80` right after you click **Evaluate**, and send me the stack trace.
2. In the browser, press F12 → Network, click Evaluate again, and open the red request to see its response body.
3. Make sure you browse to exactly `https://keycloak.lab.local`, the same name as `KC_HOSTNAME`. Using an IP or another hostname makes admin console API calls fail with generic errors.
4. Select a user in the **User** box before clicking. If that user comes from AD, Evaluate also contacts the DC, so the group mapper and LDAPS connection have to be working. A bad mapper setting (for example the wrong Groups DN) shows up as this same unknown error.

Send me the output of Step 1 and Step 2, plus the log from Step 4 if the error persists, and I'll tell you which case it is.

> **You** · Thu 15 Oct 2026, 04:48
> *(attached excerpt: "Username LDAP attribute sAMAccountName / RDN LDAP attribute cn")*
> i want to change these settings and i want to use userPrincipalName as login name. Suggest me the changes

CLAUDE

Only one of those two settings needs to change.

Setting	Current	New
Username LDAP attribute	`sAMAccountName`	**`userPrincipalName`**
RDN LDAP attribute	`cn`	**keep `cn`**
UUID LDAP attribute	`objectGUID`	keep

The RDN attribute is the one that names the user's entry in the directory tree (`CN=John Doe,OU=Users,...`). In AD that is always `cn`, and it has nothing to do with the login name. Changing it to the UPN would only break lookups.

Steps

1. Check your AD users have a UPN. Run this on a DC:

```
Get-ADUser -Filter * -Properties userPrincipalName |
  Where-Object { -not $_.userPrincipalName } | Select SamAccountName
```

Any account listed won't import into Keycloak after the change. Fix them in AD, or skip them with the filter in step 3.

2. Change the username attribute.

Go to **User federation → ad-lab → Settings → LDAP attributes** and set **Username LDAP attribute** to `userPrincipalName`. Leave RDN as `cn` and UUID as `objectGUID`.

3. (Optional) Tighten the user filter. In **Custom user LDAP filter**, add the UPN condition so only accounts with a UPN are imported:

```
(&(objectCategory=person)(objectClass=user)(userPrincipalName=*)(!(userAccountControl:1.2.840.113556.1.4.803:=2)))
```

4. Check the `username` mapper. Open the **Mappers** tab and open the mapper named `username`. Its **LDAP attribute** must be `userPrincipalName`, and if it still says `sAMAccountName`, change it and save. Keycloak normally updates it for you, but I haven't verified that on every version, so check.

Leave the `email` mapper on `mail`. Only point it at `userPrincipalName` if your users have no `mail` value in AD.

5. Clear the previously imported users. Users already imported keep their old usernames (`jdoe`), which can cause duplicates or "user exists" errors. On the provider page, choose **Action → Remove imported**. Then run **Action → Sync all users**. This is safe because AD is the source of truth and the federation is read-only.

6. Test.

- **Users:** search for `jdoe@lab.local`, and the username should be the full UPN.
- **Login:** sign in with `jdoe@lab.local` and the AD password.
- **Token:** use **Clients → argocd → Client scopes → Evaluate** and check that `preferred_username` is the UPN and `groups` is still present.

Things to know

- **Case:** Keycloak lowercases usernames, so `JDoe@Lab.Local` becomes `jdoe@lab.local`.
- **Argo CD:** the displayed user and `preferred_username` become `jdoe@lab.local`. Your group-based RBAC (`g, ArgoCD-Admins, role:admin`) is unaffected. Any per-user rules in `policy.csv` need the UPN instead.
- **Login with email:** if the UPN and `mail` are the same, users can log in either way. If you want only the UPN, switch **Login with email** off under **Realm settings → Login**.
- **Persistence:** make this change after the persistence problem from earlier is sorted. Otherwise a fresh database will discard it.

You · Thu 15 Oct 2026, 10:08

Invalid redirect URL: the protocol and host (including port) must match and the path must be within allowed URLs if provided

CLAUDE

That message comes from Argo CD, not Keycloak. Argo CD refuses to start the login because the URL in your browser doesn't match the `url` set in its config.

My values file used the placeholder `argocd.example.com`, and the chart turns that into `url: https://argocd.example.com`. If you open Argo CD any other way, such as `https://localhost:8080` through port-forward, a node IP, a NodePort or a different hostname, you get this error.

1. See what Argo CD expects

```
kubectl -n argocd get cm argocd-cm -o jsonpath='{.data.url}{"\n"}{.data.additionalUrls}{"\n"}'
```

2. Make it match the URL you actually open

Scheme, host and port must be identical to the address bar. In your values file:

```
global:
  domain: argocd.lab.local          # the real hostname you browse to

configs:
  cm:
    url: https://argocd.lab.local
    additionalUrls: |
      - https://localhost:8080      # optional: also allow port-forward access
```

Apply it:

```
helm upgrade --install argocd argo/argo-cd -n argocd \
  -f argocd-values-advanced.yaml \
  --set-string configs.secret.extra."oidc\.keycloak\.clientSecret"='<client-secret>'
kubectl -n argocd rollout restart deploy argocd-server
```

If you're testing with port-forward only, set `url: https://localhost:8080` instead (http vs https must also match).

3. Keep the Keycloak client in sync

In **Clients → argocd**, the redirect URIs must use the same base URL:

- Valid redirect URIs: `https://argocd.lab.local/auth/callback`
- Valid post logout redirect URIs: `https://argocd.lab.local/applications`
- Web origins: `https://argocd.lab.local`

If you also use `https://localhost:8080`, add the same three entries for that host. For `argocd login --sso`, also add `http://localhost:8085/auth/callback`.

How to tell the two errors apart

Error text	Where it appears	Fix
"Invalid redirect URL: the protocol and host (including port) must match..."	Argo CD page, before reaching Keycloak	`url` / `additionalUrls` in `argocd-cm` (this one)
"Invalid parameter: redirect_uri"	Keycloak page	Client's Valid redirect URIs

After the change, open the exact URL from `argocd-cm`, in a private window to avoid cached redirects, and click **Log in via Keycloak**. If it still fails, send me the address bar URL and the output of step 1.

> **You** · Thu 15 Oct 2026, 10:11
> my keyclock url is https://keycloak.vkslab.internal:8443. I am able to open the argocd console and when i click on login via keycloak. i get the above error

CLAUDE

Your Keycloak address isn't what this error is about. It comes from Argo CD and compares the address you used to open Argo CD with its configured `url`. So what I need is the Argo CD address in your browser's address bar, which you haven't given me yet.

1. Compare the two addresses

```
kubectl -n argocd get cm argocd-cm -o jsonpath='{.data.url}{"\n"}'
```

If the output differs from your browser address in scheme, hostname or port, that's the bug. For example, the config says `https://argocd.example.com` (the placeholder from my file) while you open `https://10.x.x.x:30443` or a port-forward. Set it to exactly what you open:

```
configs:
  cm:
    url: https://<exactly-what-you-type-in-the-browser>
```

Then apply it and restart the server:

```
helm upgrade --install argocd argo/argo-cd -n argocd -f argocd-values-advanced.yaml
kubectl -n argocd rollout restart deploy argocd-server
```

2. Fix the Keycloak settings for the `:8443` URL

Since Keycloak is at `https://keycloak.vkslab.internal:8443`, a few things I gave you earlier (written for `keycloak.lab.local` on port 443) need updating, or you'll hit the next error right after this one.

Compose file:

```
      KC_HOSTNAME: https://keycloak.vkslab.internal:8443
    ports:
      - "8443:8443"
```

Certificate: the hostname must be in the SAN, so if your current certificate was issued for another name, regenerate it:

```
openssl req -x509 -newkey rsa:4096 -sha256 -days 825 -nodes \
  -keyout certs/tls.key -out certs/tls.crt \
  -subj "/CN=keycloak.vkslab.internal" \
  -addext "subjectAltName=DNS:keycloak.vkslab.internal"
```

Argo CD `oidc.config`, where the issuer must include the port and match exactly:

```
  oidc.config: |
    name: Keycloak
    issuer: https://keycloak.vkslab.internal:8443/realms/lab
```

```
    clientID: argocd
    clientSecret: $oidc.keycloak.clientSecret
    requestedScopes: ["openid", "profile", "email", "groups"]
    rootCA: |
      -----BEGIN CERTIFICATE-----
      ...contents of certs/tls.crt (or your CA)...
      -----END CERTIFICATE-----
```

Keycloak client `argocd`, using your Argo CD address from step 1:

- Valid redirect URIs: `https://<argocd-address>/auth/callback`
- Valid post logout redirect URIs: `https://<argocd-address>/applications`
- Web origins: `https://<argocd-address>`

3. Check the Argo CD pods can reach Keycloak

The pods must resolve and reach the Keycloak name, not just your laptop:

```
kubectl run dnstest --rm -it --image=busybox:1.36 --restart=Never -- nslookup keycloak.vkslab.internal
curl -k https://keycloak.vkslab.internal:8443/realms/lab/.well-known/openid-configuration
```

If `nslookup` fails inside the cluster, add a CoreDNS forwarder for `vkslab.internal` pointing at your DNS server, or use `hostAliases` on the `argocd-server` pod.

Send me the `argocd-cm` URL from step 1 and your browser's Argo CD address, and I'll confirm whether that is the mismatch.

> **You** · Thu 15 Oct 2026, 12:38
> i want to expose argocd using the cilium gatewayapi share the steps to expose the same

CLAUDE

I'll put the manifests in files, since pasting YAML from chat caused trouble earlier, and validate them before sending.

📎 **argocd-gateway.yaml** and **cilium-lb-pool.yaml** attached (full contents in Appendices C and D).

I've attached `argocd-gateway.yaml` and `cilium-lb-pool.yaml`. Both parse as valid YAML, but I haven't applied them to a cluster. The design is to terminate TLS at the Cilium Gateway and send plain HTTP to `argocd-server`. I used `argocd.vkslab.internal`, so change it if your hostname differs.

1. Check the prerequisites

Cilium's Gateway API support requires kube-proxy replacement, and the Gateway API CRDs must exist.

```
kubectl get crd gateways.gateway.networking.k8s.io
cilium config view | grep -E "kube-proxy-replacement|enable-gateway-api"
```

If the CRDs are missing, install the version your Cilium release documents (check the Cilium docs for your version). For example:

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.3.0/standard-install.yaml
```

2. Enable Gateway API in Cilium

```
helm upgrade cilium cilium/cilium -n kube-system --reuse-values \
  --set kubeProxyReplacement=true \
  --set gatewayAPI.enabled=true
kubectl -n kube-system rollout restart deploy/cilium-operator ds/cilium
kubectl get gatewayclass cilium      # ACCEPTED should be True
```

If Cilium on your VKS cluster is managed through the cluster's own CNI configuration, make this change there, or a later reconcile may revert it.

3. Make sure the Gateway can get an IP

The Gateway creates a `LoadBalancer` service, which needs an address.

- **Platform load balancer:** if your environment already provides one (NSX/AVI or MetalLB), skip this step.
- **Cilium LB-IPAM:** edit the range in `cilium-lb-pool.yaml` to free addresses on your network, then apply it. You also need to enable L2 announcements (`--set l2announcements.enabled=true`) so the IP is reachable on a flat network. Newer Cilium releases may want `cilium.io/v2` instead of `v2alpha1`.

```
kubectl apply -f cilium-lb-pool.yaml
```

4. Create the TLS certificate

Lab version with a self-signed certificate:

```
openssl req -x509 -newkey rsa:4096 -sha256 -days 825 -nodes \
  -keyout argocd.key -out argocd.crt \
  -subj "/CN=argocd.vkslab.internal" \
  -addext "subjectAltName=DNS:argocd.vkslab.internal"
kubectl -n argocd create secret tls argocd-tls --cert=argocd.crt --key=argocd.key
```

For an internal CA, use cert-manager instead. Enable its Gateway API support, annotate the Gateway with `cert-manager.io/cluster-issuer: <your-issuer>`, and it will create `argocd-tls` for you.

5. Point Argo CD at the new address

In your values file, turn off the old ingress and set the public URL. This is the same `url` that caused your earlier "Invalid redirect URL" error, so it must exactly match what you type in the browser:

```
global:
  domain: argocd.vkslab.internal
configs:
  params:
    server.insecure: true     # required, the Gateway terminates TLS
  cm:
    url: https://argocd.vkslab.internal
server:
  ingress:
    enabled: false
```

```
helm upgrade --install argocd argo/argo-cd -n argocd -f argocd-values-advanced.yaml
kubectl -n argocd rollout restart deploy argocd-server
```

6. Apply the Gateway and routes

```
kubectl apply -f argocd-gateway.yaml
kubectl -n argocd get gateway argocd-gw        # PROGRAMMED True, ADDRESS filled
kubectl -n argocd get httproute
```

7. Add DNS

Take the `ADDRESS` from the Gateway and create an A record. On your AD DNS server:

```
Add-DnsServerResourceRecordA -Name argocd -ZoneName vkslab.internal -IPv4Address <gateway-address>
```

8. Update Keycloak and test

In **Clients → argocd** in Keycloak, set:

- Valid redirect URIs: `https://argocd.vkslab.internal/auth/callback`
- Valid post logout redirect URIs: `https://argocd.vkslab.internal/applications`
- Web origins: `https://argocd.vkslab.internal`

Then test:

```
curl -kI https://argocd.vkslab.internal
argocd login argocd.vkslab.internal --sso --grpc-web
```

`--grpc-web` is needed because the HTTPRoute carries the API as HTTP rather than native gRPC.

If it doesn't work

Symptom	Likely cause
GatewayClass not accepted	CRDs missing, kube-proxy replacement off, or operator not restarted
Gateway has no ADDRESS	No load balancer or IP pool, or L2 announcements not enabled
HTTPRoute not Accepted	`sectionName` or hostname doesn't match a Gateway listener
TLS listener invalid	Secret `argocd-tls` is missing, in the wrong namespace, or not type `kubernetes.io/tls`
Redirect loop or "too many redirects"	`server.insecure: true` is not applied
"Invalid redirect URL" in Argo CD	`configs.cm.url` doesn't match the browser address

`kubectl -n argocd describe gateway argocd-gw` and `describe httproute argocd` show the exact condition, so send me that output if a step fails.

> **You** · Thu 15 Oct 2026, 13:17
> how do i extract this page as a pdf

CLAUDE

The quickest way is to print the page to PDF from your browser:

1. Press **Ctrl+P** (Windows/Linux) or **Cmd+P** (Mac).
2. Set **Destination** to **Save as PDF**.
3. Open **More settings** and turn on **Background graphics**, so code blocks and tables keep their shading.
4. Click **Save**.

Printing a long chat can cut code blocks across pages or include the sidebar. If the layout looks messy, select only the answer you want first. Print dialogs only print the selection if you choose "Selection" where your browser offers it.

I can also generate a clean, formatted PDF for you instead. Tell me which you'd like:

- **Cilium Gateway API guide:** just the steps to expose Argo CD, with the commands and the two manifests.
- **Full runbook:** Argo CD Helm install, Keycloak in Docker, AD LDAPS and UPN login, OIDC integration, and the Gateway exposure, in one document.

> **You** · Thu 15 Oct 2026, 13:19
> export this complete page as pdf from top to bottom

CLAUDE

You're reading this PDF: the complete conversation, top to bottom. The four files attached along the way follow as appendices.

Appendices

Appendix A: argocd-values-advanced.yaml

Advanced Argo CD Helm values (HA, sharded controller, SSO, metrics, notifications)

```yaml
# Advanced / production Argo CD values (argo/argo-cd Helm chart)
# Always validate against your pinned chart version:
#   helm show values argo/argo-cd --version <ver> > defaults.yaml
#
# Prereqs assumed: ingress-nginx, cert-manager (ClusterIssuer), kube-prometheus-stack,
# External Secrets Operator (or Sealed Secrets), >= 3 schedulable nodes (HA Redis, hard anti-affinity).

global:
  domain: argocd.example.com
  logging:
    format: json
    level: info
  # Never co-locate replicas of the same component on one node
  affinity:
    podAntiAffinity: hard
  # Restrict traffic between Argo CD components (needs a CNI that enforces NetworkPolicy)
  networkPolicy:
    create: true

crds:
  install: true
  keep: true            # helm uninstall will NOT delete CRDs (and your Applications)

# ------------------------------------------------------------------ configs
configs:
  params:
    server.insecure: true                       # TLS terminated at the ingress
    controller.status.processors: 50
    controller.operation.processors: 25
    controller.repo.server.timeout.seconds: 120
    controller.self.heal.timeout.seconds: 5
    reposerver.parallelism.limit: 10            # cap concurrent manifest generations
    application.namespaces: "team-*"            # "apps in any namespace" (also needs AppProject
sourceNamespaces)
    applicationsetcontroller.enable.progressive.syncs: true

  cm:
    timeout.reconciliation: 180s                # default 120s; add jitter below
    timeout.reconciliation.jitter: 60s
    application.resourceTrackingMethod: annotation
    exec.enabled: false                         # disable web terminal
    admin.enabled: false                        # set to true until SSO is verified, then flip to false
    statusbadge.enabled: true
    server.rbac.log.enforce.enable: true

    # --- Direct OIDC (Keycloak/Okta/Entra/etc.), Dex disabled below ---
    oidc.config: |
      name: SSO
```

```
      issuer: https://sso.example.com/realms/platform
      clientID: argocd
      clientSecret: $oidc.sso.clientSecret       # key inside the argocd-secret (see configs.secret.extra)
      requestedScopes: ["openid", "profile", "email", "groups"]

    # --- Keep noisy / generated resources out of the UI and diffing ---
    resource.exclusions: |
      - apiGroups: ["cilium.io"]
        kinds: ["CiliumIdentity"]
        clusters: ["*"]
      - apiGroups: ["", "discovery.k8s.io"]
        kinds: ["Endpoints", "EndpointSlice"]
        clusters: ["*"]
    resource.customizations.ignoreDifferences.apps_Deployment: |
      jqPathExpressions:
        - .spec.replicas        # let HPA own replicas

  rbac:
    policy.default: role:readonly
    scopes: "[groups]"
    policy.csv: |
      p, role:deployer, applications, sync,   */*, allow
      p, role:deployer, applications, get,    */*, allow
      p, role:deployer, applications, action/*, */*, allow
      g, platform-admins, role:admin
      g, app-deployers,   role:deployer

  # Secrets: do NOT keep real values here. Populate argocd-secret keys via ESO/Sealed Secrets
  # and let the chart skip creating them.
  secret:
    createSecret: true
    # extra:
    #   oidc.sso.clientSecret: <injected externally>

  # Repo credential templates are better managed as ExternalSecrets with label
  # argocd.argoproj.io/secret-type: repo-creds (or repository)

# ------------------------------------------------------------------- application controller (sharded)
controller:
  replicas: 2                      # chart sets ARGOCD_CONTROLLER_REPLICAS automatically -> cluster sharding
  resources:
    requests: { cpu: 500m, memory: 1Gi }
    limits:   { memory: 2Gi }
  pdb:
    enabled: true
    minAvailable: 1
  metrics:
    enabled: true
    serviceMonitor:
      enabled: true
      additionalLabels: { release: kube-prometheus-stack }
    rules:
      enabled: true
      spec:
        - alert: ArgoAppOutOfSync
          expr: argocd_app_info{sync_status="OutOfSync"} == 1
```

```
          for: 30m
          labels: { severity: warning }
        - alert: ArgoAppUnhealthy
          expr: argocd_app_info{health_status!~"Healthy|Progressing"} == 1
          for: 15m
          labels: { severity: critical }

# ------------------------------------------------------------------ API server / UI
server:
  autoscaling:
    enabled: true
    minReplicas: 2
    maxReplicas: 5
    targetCPUUtilizationPercentage: 70
  resources:
    requests: { cpu: 100m, memory: 256Mi }
    limits:   { memory: 512Mi }
  pdb:
    enabled: true
    minAvailable: 1
  metrics:
    enabled: true
    serviceMonitor:
      enabled: true
      additionalLabels: { release: kube-prometheus-stack }
  ingress:
    enabled: true
    ingressClassName: nginx
    tls: true                                  # cert-manager issues argocd-server-tls
    annotations:
      cert-manager.io/cluster-issuer: letsencrypt-prod
      nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
      nginx.ingress.kubernetes.io/backend-protocol: "HTTP"
  # No LoadBalancer needed when behind ingress. On clusters without an ingress controller use:
  # service:
  #   type: LoadBalancer

# ------------------------------------------------------------------ repo server
repoServer:
  autoscaling:
    enabled: true
    minReplicas: 2
    maxReplicas: 6
    targetCPUUtilizationPercentage: 70
  resources:
    requests: { cpu: 250m, memory: 512Mi }
    limits:   { memory: 1Gi }
  pdb:
    enabled: true
    minAvailable: 1
  metrics:
    enabled: true
    serviceMonitor:
      enabled: true
      additionalLabels: { release: kube-prometheus-stack }
  # Needed if you use Helm/Kustomize plugins or write caches under a read-only root FS
```

```yaml
    volumes:
      - name: tmp
        emptyDir: {}
    volumeMounts:
      - name: tmp
        mountPath: /tmp

  # ------------------------------------------------------------- ApplicationSet controller
  applicationSet:
    replicas: 2
    pdb:
      enabled: true
      minAvailable: 1
    metrics:
      enabled: true
      serviceMonitor:
        enabled: true
        additionalLabels: { release: kube-prometheus-stack }

  # ------------------------------------------------------------- Notifications
  notifications:
    enabled: true
    metrics:
      enabled: true
      serviceMonitor:
        enabled: true
        additionalLabels: { release: kube-prometheus-stack }
    secret:
      create: false                              # provide argocd-notifications-secret via ESO (key: slack-
token)
    notifiers:
      service.slack: |
        token: $slack-token
    subscriptions:
      - recipients: ["slack:platform-deployments"]
        triggers: [on-sync-failed, on-health-degraded, on-sync-succeeded]
    templates:
      template.app-sync-failed: |
        message: |
          :x: *{{.app.metadata.name}}* sync failed.
          {{.app.status.operationState.message}}
          <{{.context.argocdUrl}}/applications/{{.app.metadata.name}}|Open in Argo CD>
    triggers:
      trigger.on-sync-failed: |
        - when: app.status.operationState.phase in ['Error', 'Failed']
          send: [app-sync-failed]

  # ------------------------------------------------------------- Dex off (using direct OIDC), HA Redis on
  dex:
    enabled: false

  redis:
    enabled: false

  redis-ha:
    enabled: true
```

```
  haproxy:
    enabled: true
    metrics:
      enabled: true
  persistentVolume:
    enabled: false          # cache only; safe to lose
  hardAntiAffinity: true    # requires >= 3 nodes
  metrics:
    enabled: true

redisSecretInit:
  enabled: true
```

Appendix B: docker-compose-persistent.yml

Keycloak + Postgres with pinned persistent volumes and daily backup

```
name: keycloak

services:
  postgres:
    image: postgres:17
    container_name: keycloak-db
    restart: unless-stopped
    environment:
      POSTGRES_DB: keycloak
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U keycloak -d keycloak"]
      interval: 10s
      timeout: 5s
      retries: 5
  keycloak:
    image: quay.io/keycloak/keycloak:26.4
    container_name: keycloak
    restart: unless-stopped
    command: start
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      KC_DB: postgres
      KC_DB_URL: jdbc:postgresql://postgres:5432/keycloak
      KC_DB_USERNAME: keycloak
      KC_DB_PASSWORD: ${POSTGRES_PASSWORD}
      KC_HOSTNAME: https://keycloak.lab.local
      KC_HTTPS_CERTIFICATE_FILE: /opt/keycloak/conf/certs/tls.crt
      KC_HTTPS_CERTIFICATE_KEY_FILE: /opt/keycloak/conf/certs/tls.key
      KC_TRUSTSTORE_PATHS: /opt/keycloak/conf/truststores
      KC_HEALTH_ENABLED: "true"
      KC_METRICS_ENABLED: "true"
      KC_BOOTSTRAP_ADMIN_USERNAME: admin
```

```
      KC_BOOTSTRAP_ADMIN_PASSWORD: ${KEYCLOAK_ADMIN_PASSWORD}
    ports:
      - "443:8443"
    volumes:
      - ./certs:/opt/keycloak/conf/certs:ro
      - ./truststore:/opt/keycloak/conf/truststores:ro
      - kcdata:/opt/keycloak/data
  db-backup:
    image: postgres:17
    container_name: keycloak-db-backup
    restart: unless-stopped
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      PGHOST: postgres
      PGUSER: keycloak
      PGPASSWORD: ${POSTGRES_PASSWORD}
      PGDATABASE: keycloak
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        while true; do
          pg_dump -Fc -f /backups/keycloak-$$(date +%Y%m%d-%H%M).dump
          find /backups -name 'keycloak-*.dump' -mtime +7 -delete
          sleep 86400
        done
    volumes:
      - ./backups:/backups
volumes:
  pgdata:
    name: keycloak_pgdata
  kcdata:
    name: keycloak_data
```

Appendix C: argocd-gateway.yaml

Cilium Gateway API: Gateway, HTTPS route, HTTP-to-HTTPS redirect

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: argocd-gw
  namespace: argocd
spec:
  gatewayClassName: cilium
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      hostname: argocd.vkslab.internal
      allowedRoutes:
        namespaces:
          from: Same
    - name: https
```

```
      protocol: HTTPS
      port: 443
      hostname: argocd.vkslab.internal
      tls:
        mode: Terminate
        certificateRefs:
          - kind: Secret
            name: argocd-tls
      allowedRoutes:
        namespaces:
          from: Same
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: argocd-http-redirect
  namespace: argocd
spec:
  parentRefs:
    - name: argocd-gw
      sectionName: http
  hostnames:
    - argocd.vkslab.internal
  rules:
    - filters:
        - type: RequestRedirect
          requestRedirect:
            scheme: https
            statusCode: 301
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: argocd
  namespace: argocd
spec:
  parentRefs:
    - name: argocd-gw
      sectionName: https
  hostnames:
    - argocd.vkslab.internal
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: argocd-server
          port: 80
```

Appendix D: cilium-lb-pool.yaml

Optional Cilium LB-IPAM pool and L2 announcement policy

```yaml
apiVersion: cilium.io/v2alpha1
kind: CiliumLoadBalancerIPPool
metadata:
  name: lab-pool
spec:
  blocks:
    - start: "192.168.10.200"
      stop: "192.168.10.220"
---
apiVersion: cilium.io/v2alpha1
kind: CiliumL2AnnouncementPolicy
metadata:
  name: lab-l2
spec:
  loadBalancerIPs: true
  interfaces:
    - ^eth[0-9]+
```